

Homework-5

① Initially, at step-0 we have 0 edges in minimum spanning tree (MST) and visited vertices $V = \{v_1\}$. At each step we choose the edge with the lowest weight that connects a vertex already in the tree to a new vertex. Prim's algorithm

Step-0:

$$\text{MST} = \emptyset = \{ \}$$

$$V = \{v_1\}$$

Step-1:

$v_1 \xrightarrow{17} v_4$, $v_1 \xrightarrow{32} v_2$. We choose v_4 and add it.

$$V = \{v_1, v_4\}$$

$$\text{MST} = \{(v_1, v_4)\}$$

Step-2:

$v_4 \xrightarrow{3} v_8$, $v_4 \xrightarrow{10} v_5$, $v_4 \xrightarrow{18} v_3$, $v_1 \xrightarrow{32} v_2$. We choose minimum which is v_8 .

$$V = \{v_1, v_4, v_8\}$$

$$\text{MST} = \{(v_1, v_4), (v_4, v_8)\}$$

Step-3:

$v_8 \xrightarrow{4} v_9$, $v_4 \xrightarrow{10} v_5$, $v_4 \xrightarrow{18} v_3$, $v_1 \xrightarrow{32} v_2$, $v_8 \xrightarrow{59} v_7$

We choose minimum v_9 .

$$V = \{v_1, v_4, v_8, v_9\}$$

$$\text{MST} = \{(v_1, v_4), (v_4, v_8), (v_8, v_9)\}$$

Step-4 :

$V_4 \xrightarrow{10} V_5$, $V_9 \xrightarrow{12} V_{10}$, $V_4 \xrightarrow{18} V_3$, $V_9 \xrightarrow{25} V_5$, $V_1 \xrightarrow{32} V_2$,

$V_8 \xrightarrow{59} V_7$.. we choose the minimum weight one V_5 .

$$V = \{V_1, V_4, V_8, V_9, V_5\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5)\}$$

Step-5 :

$V_9 \xrightarrow{12} V_{10}$, $V_4 \xrightarrow{18} V_3$, $V_5 \xrightarrow{28} V_8$, $V_1 \xrightarrow{32} V_2$,

$V_5 \xrightarrow{45} V_2$, $V_8 \xrightarrow{59} V_7$.. we choose V_{10} .

$$V = \{V_1, V_4, V_8, V_9, V_5, V_{10}\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5), (V_9, V_{10})\}$$

Step-6 :

$V_{10} \xrightarrow{6} V_6$, $V_4 \xrightarrow{18} V_3$, $V_5 \xrightarrow{28} V_6$, $V_1 \xrightarrow{32} V_2$, $V_5 \xrightarrow{45} V_2$,

$V_8 \xrightarrow{59} V_7$.. we choose minimum V_6 .

$$V = \{V_1, V_4, V_8, V_9, V_5, V_{10}, V_6\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5), (V_9, V_{10}), (V_{10}, V_6)\}$$

Step-7 :

$V_4 \xrightarrow{18} V_3$, $V_1 \xrightarrow{32} V_2$, $V_5 \xrightarrow{45} V_2$, $V_8 \xrightarrow{59} V_7$.. We choose minimum V_3 .

$$V = \{V_1, V_4, V_8, V_9, V_5, V_{10}, V_6, V_3\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5), (V_9, V_{10}), (V_{10}, V_6), (V_4, V_3)\}$$

Step-8 :

$V_3 \xrightarrow{5} V_7, V_1 \xrightarrow{32} V_2, V_5 \xrightarrow{45} V_2, V_8 \xrightarrow{59} V_7$, We

choose V_7 .

$$V = \{V_1, V_4, V_8, V_9, V_5, V_{10}, V_6, V_3, V_7\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5), (V_9, V_{10}), (V_{10}, V_6), (V_4, V_3), (V_3, V_7)\}$$

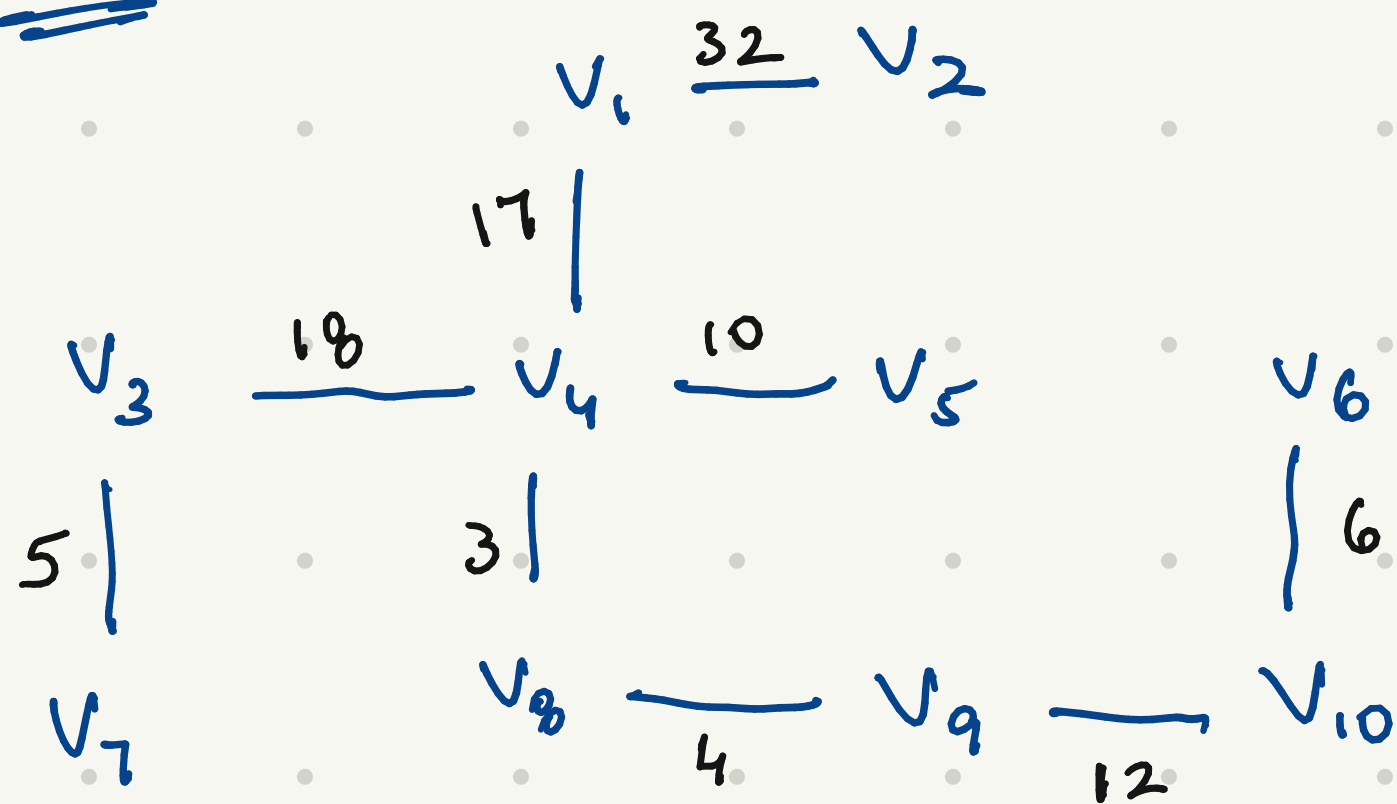
Step-9 :

$V_1 \xrightarrow{32} V_2, V_5 \xrightarrow{45} V_2$. We choose minimum V_2 .

$$V = \{V_1, V_4, V_8, V_9, V_5, V_{10}, V_6, V_3, V_7, V_2\}$$

$$MST = \{(V_1, V_4), (V_4, V_8), (V_8, V_9), (V_4, V_5), (V_9, V_{10}), (V_{10}, V_6), (V_4, V_3), (V_3, V_7), (V_1, V_2)\}$$

MST:-



② Kruskal's algorithm for above question

First, we sort all edges by weight and add the smallest available without creating cycles.

$$(V_4, V_8) - 3$$

$$(V_3, V_4) - 18$$

$$(V_8, V_9) - 4$$

$$(V_5, V_9) - 25$$

$$(V_3, V_7) - 5$$

$$(V_5, V_6) - 28$$

$$(V_6, V_{10}) - 6$$

$$(V_1, V_2) - 32$$

$$(V_4, V_5) - 10$$

$$(V_2, V_5) - 45$$

$$(V_9, V_{10}) - 12$$

$$(V_7, V_8) - 59$$

$$(V_1, V_4) - 17$$

Step-1: add (V_4, V_8) with weight 3

$$MST = \{ (V_4, V_8) \}$$

Step-2: $(V_8, V_9) - 4$

$$MST = \{ (V_4, V_8), (V_8, V_9) \}$$

Step-3: $(V_3, V_7) - 5$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7) \}$$

Step-4: $(V_6, V_{10}) - 6$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}) \}$$

Step-5: $(V_4, V_5) - 10$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5) \}$$

Step-6: $(V_9, V_{10}) - 12$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}) \}$$

Step-7: $(V_1, V_4) - 17$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}), (V_1, V_4) \}$$

Step-8: $(V_3, V_4) - 18$

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}), (V_1, V_4), \\ (V_3, V_4) \}$$

Step-9: $(V_5, V_9) - 25$, we don't add because it creates cycles.

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}), (V_1, V_4), \\ (V_3, V_4) \}$$

Step-10: $(V_5, V_6) - 28$, we don't add because it creates cycle.

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}), \\ (V_1, V_4), (V_3, V_4) \}$$

Step-11: $(V_1, V_2) - 32$ add it

$$MST = \{ (V_4, V_8), (V_8, V_9), (V_3, V_7), (V_6, V_{10}), (V_4, V_5), (V_9, V_{10}), \\ (V_1, V_4), (V_3, V_4), (V_1, V_2) \}$$

Now the MST connects all the 10 vertices with 9 edges. So, we don't evaluate (V_2, V_5) & (V_7, V_8) . We terminate early.

MST:

$$(V_4, V_8) - 3$$

$$(V_8, V_9) - 4$$

$$(V_3, V_7) - 5$$

$$(V_6, V_{10}) - 6$$

$$(V_4, V_5) - 10$$

$$(V_9, V_{10}) - 12$$

$$(V_1, V_4) - 17$$

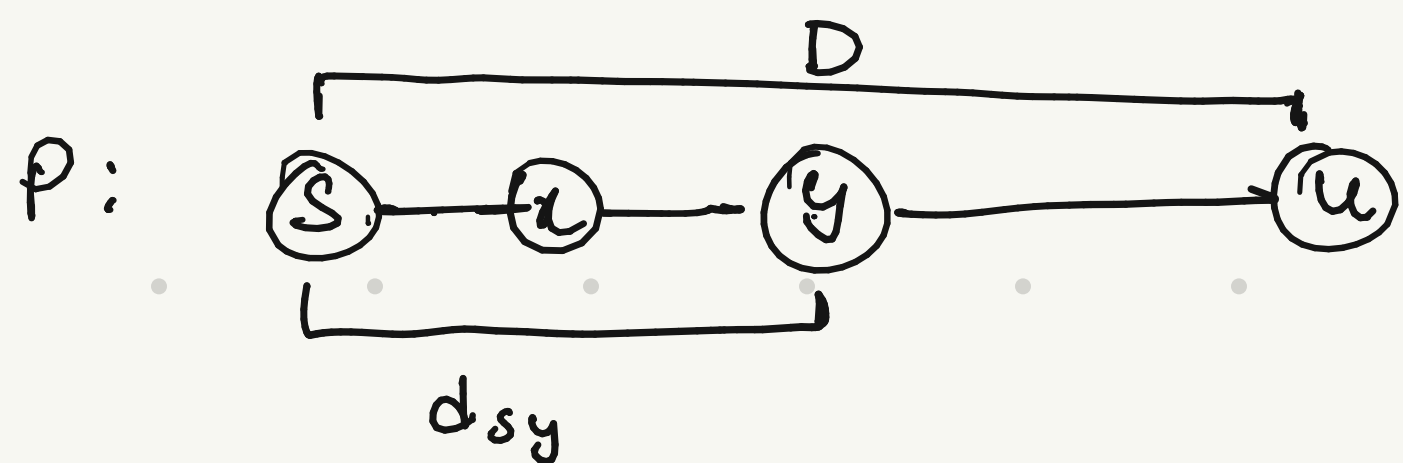
$$(V_3, V_4) - 18$$

$$(V_1, V_2) - 32$$

- ③ We need to prove that Dijkstra's algorithm correctly finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative edge weights.

Let,

- $w(u,v)$ be the weight of edge (u,v) with $w(u,v) \geq 0$.
 - $\text{dist}[v]$ be the shortest distance found so far from s to v .
 - S : set of vertices whose final shortest distance from s has been computed.
 - s is the source vertex.
1. At induction step- k , the beginning of the iteration of Dijkstra's algorithm in which k^{th} vertex is added to S , the following holds:-
- For every vertex $v \in S$ $\text{dist}[v]$ is the true shortest path distance from s to v .
2. Base case ($k=1$): S contains only source s . By initialization, $\text{dist}[s] = 0$ which is the true shortest distance to itself.
3. Lets assume the hypothesis is true just before we add the k -vertex to S and u is the vertex to be added in k^{th} iteration. We need to show that
- $\text{dist}[u]$ is the true shortest path distance from s to u .
 - After adding u to S , the hypothesis is true for next step.
4. At the moment u is chosen, $\text{dist}[u]$ is the shortest path length from s to u . For contradiction, there is a path P from s to u with length $D < \text{dist}[u]$.
- Let y be the first vertex along P that is not in S .
 - Let x be the vertex in S before y on P .



$$d_{sy} < D < \text{dist}[u]$$

5. We assumed $x \in S$ so $\text{dist}[x]$ is true shortest distance to x .

$$\text{dist}[x] \leq \text{length}(s \rightarrow x \text{ along } P)$$

edge (x, y) has weight $w(x, y) \geq 0$, then distance to y is at most.

$$\text{dist}[x] + w(x, y) \leq \text{length}(s \rightarrow y \text{ along } P) < \text{dist}[u]$$

6. But Dijkstra's already satisfied when x is added to S , so when u is chosen $\text{dist}[y]$ already satisfies.

$\text{dist}[y] \leq \text{dist}[x] + w(x, y) < \text{dist}[u] \rightarrow$ this contradicts the choice of u not in S with smallest distance.

7. Therefore, no path P exists that is shorter. So, $\text{dist}[u]$ is the true shortest distance to u .

8. After u is added to S , we consider each neighbor v of u not in S .

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + w(u, v))$$

This maintains the property that $\text{dist}[v]$ is shortest distance from s to v using only vertices in S as intermediate nodes. The induction hypothesis $k+1$ holds true: all vertices in S including u now have the correct shortest distances.

④ 0-1 Knapsack problem :-

Given n items, i th item has value v_i with weight w_i .
Find a knapsack with capacity W where each item can be taken atmost once (0-1). The goal is to maximize total value without exceeding weight capacity.

$K(i, c)$ = maximum value achievable using first i items with capacity c .

$$K(i, c) = \begin{cases} 0 & \text{if } i=0 \text{ or } c=0 \\ K(i-1, c) & \text{if } w_i > c \\ \max(K(i-1, c), K(i-1, c-w_i) + v_i) & \text{if } w_i \leq c \end{cases}$$

$K(i-1, c) \rightarrow$ skip i th item

$K(i-1, c-w_i) + v_i \rightarrow$ pick item i

Tabulation approach: Create DP table with $n+1$ rows and $W+1$ columns

knapsack (values[1...n], weights[1...n], W) :

initialize the table with $n+1$ rows, $W+1$ cols to 0

$$K[0...n][0...W] = 0$$

fill the table

for $i = 1 \rightarrow n$:

for $c = 1 \rightarrow W$:

$$K[i][c] = K[i-1][c]$$

if $weights[i] \leq c$:

$$value_if_take = K[i-1][c - weights[i]] + values[i]$$

if $value_if_take > K[i][c]$:

$$K[i][c] = value_if_take$$

get the max value

max-value = $K[n][W]$

tracing back to find selected items

selected-items = []

rem-capacity = W

for $i = n \rightarrow 1$:

if $K[i][\text{rem-capacity}] \neq K[i-1][\text{rem-capacity}]$:

selected-items.append(i)

rem-capacity -= weights[i]

return max-value, selected-items

Time complexity = $O(n \times W)$